

Arquitectura de computadoras

Representación de la información

Dr. Rodrigo Vázquez López

Universidad Autónoma de la Ciudad de México
Plantel San Lorenzo Tezonco

Semestre 2026-I

UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO

Contenido

- 1 Conceptos básicos
- 2 Representación de la información
- 3 Números enteros
 - Representación binaria
 - Representación signo-magnitud
 - Complemento a 1
 - Complemento a 2
- 4 Números de punto flotante
 - Principios del punto flotante
 - Formato IEEE-754
- 5 Caracteres
 - ASCII
 - UNICODE



Vectores binarios

Un solo **dígito binario** se denomina **bit**. Para representar números, **agrupamos** los bits en **secuencias** más grandes las cuales denominamos **vectores binarios**.

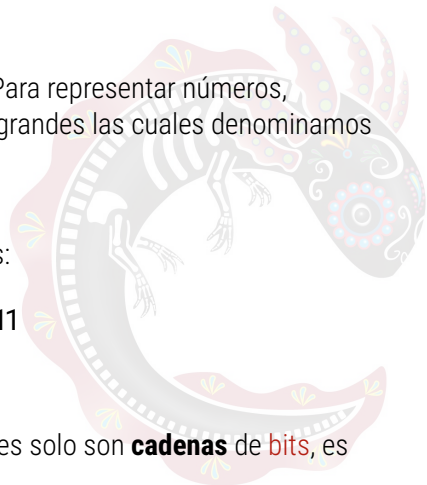
Ejemplo:

Consideramos el siguiente vector de bits:

1101111

El vector tiene **7 bits**.

Es importante mencionar que los vectores solo son **cadenas** de **bits**, es decir, no tienen **ningun significado**.

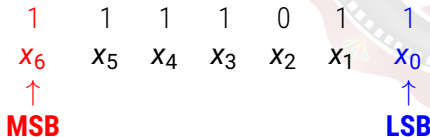


Bits más y menos significativos

Hacen referencia a los bits que tienen **mayor** y **menor** valor. Para identificarlos, se leen los bits de **derecha a izquierda**, asignando a cada bit un **índice**. El bit más a la **derecha** (bit cero o x_0) se denomina el **menos significativo (LSB)**, mientras que el bit más a la **izquierda** (bit $n - 1$ o x_{n-1}) es el **más significativo (MSB)**.

Ejemplo:

Sea el vector binario **1111011**, identifique el bit mas significativo y el menos significativo:



Conjuntos especiales de vectores binarios

Un vector de **cuatro bits** es nombrado **nibble**.

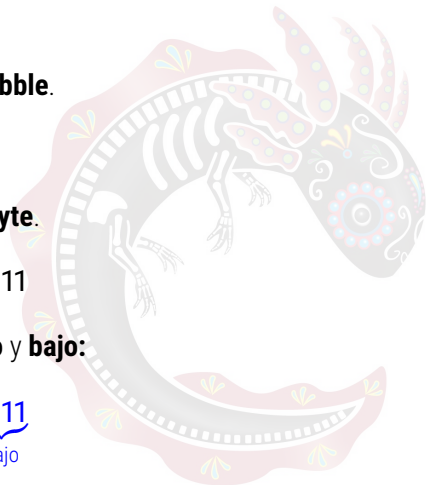
1001

Un vector de **ocho bits** se le denomina **byte**.

10101111

Un byte se compone de **dos nibbles**, **alto** y **bajo**:

1010 1111
alto bajo



Múltiplos del byte

El byte se utiliza como **medida de almacenamiento** de información, por lo que los **múltiplos** del byte son los siguientes:

Nombre	Descripción	Valor en bits
Byte (B)	Conjunto de 8 bits	8 bits
Kilobyte (KB)	Conjunto de 1024 bytes	1024×8 bits
Megabyte (MB)	Conjunto de 1024 KB	$1024^2 \times 8$ bits
Gigabyte (GB)	Conjunto de 1024 MB	$1024^3 \times 8$ bits
Terabyte (TB)	Conjunto de 1024 GB	$1024^4 \times 8$ bits
Petabyte (PB)	Conjunto de 1024 TB	$1024^5 \times 8$ bits
Exabyte (EB)	Conjunto de 1024 PB	$1024^6 \times 8$ bits

Información

La **información** es el conjunto de **datos organizados** y **procesados** que constituyen **mensajes, instrucciones, operaciones, funciones** y cualquier tipo de actividad que tenga lugar en relación con la computadora.

La información se puede **clasificar** como

- **Datos numéricos:** generalmente combinaciones de **dígitos** del 0 al 9.
- **Datos alfabéticos:** compuestos sólo por **letras**.
- **Datos alfanuméricos:** combinación de los **dos anteriores**.

Representación de la información

Dependiendo del tipo de **información**, existen diferentes formas de **representar datos** en una **computadora**:

■ Números enteros

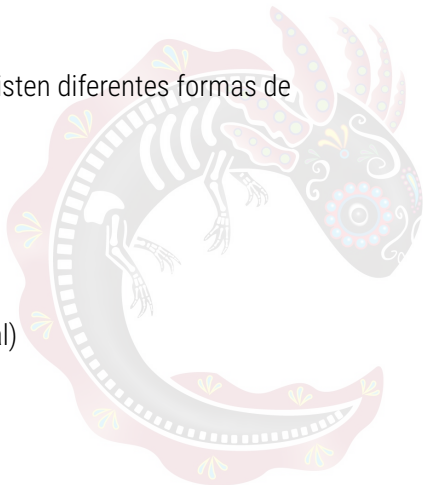
- Representación binaria
- Signo magnitud
- Complemento 1
- Complemento 2

■ Números reales (con punto decimal)

- Estándar IEEE-754

■ Caracteres alfanuméricos

- ASCII
- UNICODE



Representación binaria

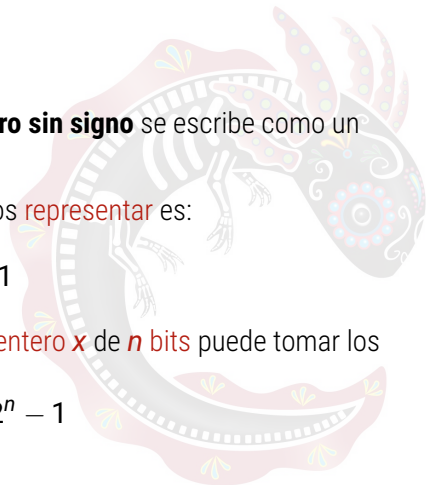
En esta representación, un **número entero sin signo** se escribe como un **número binario** de n bits.

Con n bits, el **valor máximo** que podemos **representar** es:

$$2^n - 1$$

Por lo tanto, en esta representación, un **entero x** de n bits puede tomar los **valores**:

$$0 \leq x \leq 2^n - 1$$



Representación binaria

Ejemplo

Represente el numero 57 como entero sin signo con 8 bits.

- **Verificamos** que el número pueda ser representable con 8 bits:

$$\begin{aligned}2^n - 1 &= 2^8 - 1 \\&= 256 - 1 \\&= 255\end{aligned}$$

Como $57 \leq 255$ entonces podemos representar el 57 con 8 bits.

- **Convertimos** el 57 a binario:

$$57_{10} = 111001_2$$

- Finalmente, **escribimos** el 57 con 8 bits:

$$57 = 00111001$$

Representación signo-magnitud

Sirve para representar **números entero con signo** con n bits asignando el bit **más significativo** para almacenar el **signo** y **el resto** de los bits ($n - 1$) para almacenar la **magnitud**. El **bit de signo** se establece en **1** para un **entero negativo** y **0** para un **entero positivo**. La **magnitud** se escribe como **número binario** con $n - 1$ bits.

En esta representación, un **entero x** de n bits puede tomar los **valores**:

$$-2^{n-1} - 1 \leq x \leq +2^{n-1} - 1$$

En esta representación hay **dos versiones** del **cero**, es decir, ± 0 . Por ejemplo, para 8 bits:

$$00000000 = +0$$

$$10000000 = -0$$

Representación signo-magnitud

Ejemplo:

Represente el ± 27 en formato signo-magnitud con 8 bits:

- Con **8 bits** podemos **representar**:

$$\begin{aligned} -2^{8-1} - 1 \leq x \leq +2^{8-1} - 1 &= -2^7 - 1 \leq x \leq +2^7 - 1 \\ &= -127 \leq x \leq +127 \end{aligned}$$

- **Convertimos** el 27 a **binario**:

$$27_{10} = 11011_2$$

- Finalmente, **escribimos** la magnitud con **7 bits** y agregamos los **bits de signo**:

$$\mathbf{0}0011011 = +27$$

$$\mathbf{1}0011011 = -27$$

Representación complemento a 1

En esta representación, asignamos el bit **más significativo** para almacenar el **signo** y **el resto** de los bits ($n - 1$) para almacenar la **magnitud**. Los **números positivos** se obtienen de la misma forma que en el formato **signo-magnitud** mientras que los **números negativos** se obtienen **intercambiando** **ceros por unos y unos por ceros**.

En esta representación, un **entero x** de **n bits** puede tomar los **valores**:

$$-2^{n-1} - 1 \leq x \leq +2^{n-1} - 1$$

En esta representación también hay **dos versiones** del **cero**, es decir, ± 0 .
Por ejemplo, para 8 bits:

$$00000000 = +0$$

$$10000000 = -0$$

Representación complemento a 1

Ejemplo:

Represente el ± 42 en formato complemento a 1 con 8 bits:

- Al igual que con el formato signo-magnitud, con **8 bits** podemos **representar** el rango $-127 \leq x \leq +127$
- **Convertimos** el 42 a **binario**:

$$42_{10} = 101010_2$$

- El **valor positivo** lo obtenemos escribiendo el número en binario con **7 bits** y **agregando** el **bit de signo**:

$$\mathbf{0}0101010 = +42$$

Representación complemento a 1

- Para obtener el **negativo**, escribimos el número en binario con 7 bits:

0101010

- Después, **intercambiamos** ceros por unos y unos por ceros:

0	1	0	1	0	1	0
↓	↓	↓	↓	↓	↓	↓
1	0	1	0	1	0	1

- Finalmente, **agregamos** el bit de signo:

11010101 = -42

Representación complemento a 2

Es una **extensión** de la representación **complemento a 1**. A diferencia de anterior, el rango es **asimétrico** por lo que elimina el problema de tener **dos representaciones** para el cero. Los **números positivos** se obtienen de la misma forma que complemento a 1 mientras que los **números negativos** se obtienen **realizando primero** la representación **complemento a 1** y **sumando** al resultado **una unidad**.

En esta representación, un **entero x** de **n bits** puede tomar los **valores**:

$$-2^{n-1} \leq x \leq +2^{n-1} - 1$$

Representación complemento a 2

Ejemplo:

Escriba el ± 20 en representación complemento a 2 con 8 bits:

- En este formato, con **8 bits** podemos **representar** el rango $-128 \leq x \leq +127$
- El **positivo** lo obtenemos de la **misma forma** que en **complemento a 1**:

$$00010100 = +20$$

- El **negativo** lo obtenemos realizando los **mismos pasos** que **complemento a 1** hasta **antes** de agregar el **bit de signo**, es decir, primero **escribimos** el número en binario con **7 bits**:

$$0010100$$

■ **Intercambiamos** ceros por unos y unos por ceros:

0	0	1	0	1	0	0
↓	↓	↓	↓	↓	↓	↓
1	1	0	1	0	1	1

■ Al resultado le **sumamos** la **unidad**:

	1	1	0	1	0	1	1
+							1
	1	1	0	1	1	0	0

- 0 0 1 0 1 0 0
↓ ↓ ↓ ↓ ↓ ↓ ↓
1 1 0 1 0 1 1

- $$\begin{array}{r} 1101011 \\ + \\ \hline 1101100 \end{array}$$

Representación complemento a 2

- Finalmente, **agregamos** el **bit de signo**:

$$11101100 = -20$$

- Podemos **comprobar** el resultado de la siguiente manera:

$$\begin{aligned} -20 &= -1(2^7) + 1(2^6) + 1(2^5) + 1(2^3) + 1(2^2) \\ &= -128 + 64 + 32 + 8 + 4 \\ &= -20 \end{aligned}$$

Ejercicios

Represente los siguientes números en formato **signo-magnitud**, **complemento a 1** y **complemento a 2** con 8 bits:

- ± 10
- ± 33
- ± 105
- ± 127



Principios del punto flotante

Una forma de separar el **rango** de la **precisión** es expresar los números en **notación científica**.

$$n = f \times 10^e$$

donde **f** se llama **fracción** o **mantisa** y **e** es un **entero positivo** o **negativo** llamado **exponente**. La versión computacional de esta notación se llama **punto flotante**. Algunos ejemplos de números expresados de esta forma son:

3.14	= 0.314	$\times 10^1$	= 3.14	$\times 10^0$
0.000001	= 0.1	$\times 10^{-5}$	= 1.0	$\times 10^{-6}$
1941	= 0.1941	$\times 10^4$	= 1.941	$\times 10^3$

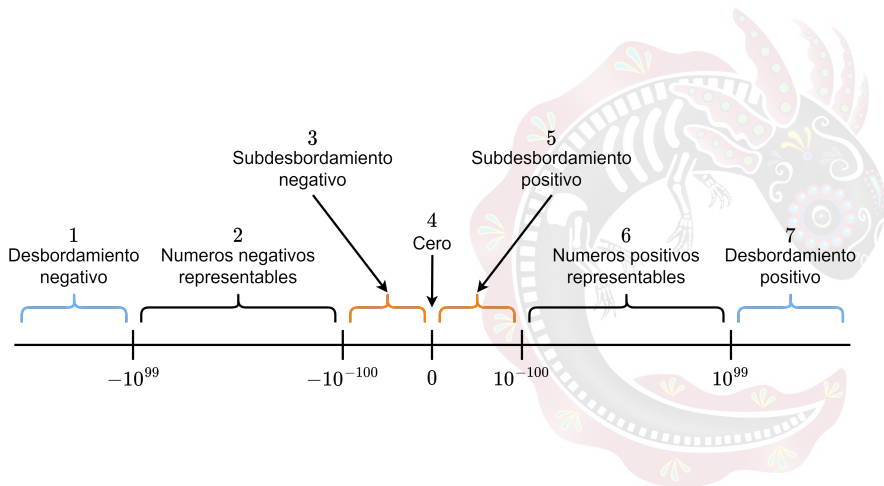
El **rango** está efectivamente determinado por el **número de dígitos** del **exponente** y la **precisión** está determinada por el **número de dígitos** de la **fracción**.

Regiones de la recta real

Los números de **punto flotante** se pueden utilizar para **modelar** los **números reales**, aunque existen algunas diferencias. La **recta real** se divide en **siete regiones**:

- 1 Números **negativos grandes menores** que -0.999×10^{99}
- 2 Números **negativos entre** -0.999×10^{99} y -0.100×10^{-99}
- 3 Números **negativos** pequeños con **magnitudes inferiores** a 0.100×10^{-99}
- 4 **Cero**
- 5 Números **positivos** pequeños con **magnitudes inferiores** a 0.100×10^{-99}
- 6 Números **positivos entre** 0.100×10^{-99} y 0.999×10^{99}
- 7 Números **positivos grandes mayores** que 0.999×10^{99}

Regiones de la recta real



Regiones de la recta real

Las **regiones 1, 3, 5 o 7** **no pueden ser representadas** mediante un formato de **punto flotante**. La razón de esto se debe a la **naturaleza finita** de la representación de los números y, por tanto, **es inevitable**.

Si el **resultado** de una **operación aritmética** produce un número en las **regiones 1 o 7** se obtiene un **error de desbordamiento**. Por otro lado, si el resultado se da en las **regiones 3 o 5** se obtiene un error de **subdesbordamiento**, el cual es **menos grave** porque 0 suele ser una aproximación satisfactoria.

Además, **no todos los números** comprendidos en las **regiones 2 y 6** **tienen representación**, es decir, **existen huecos** con números que la computadora **aproxima** al más cercano para trabajar con ellos.

Números de punto flotante

Decimos que un **número** x en **base** b está representado en **punto flotante** si se escribe de la siguiente forma:

$$x = -1^s m \cdot b^e$$

Con $m, e \in \mathbb{Z}$. A m se le llama **mantisa**, e el **exponente** y s el **bit de signo**.

De manera informal, un número de **punto flotante** tiene de la siguiente forma:

$$x = \pm \underbrace{dd \dots dd.dd \dots dd}_{p \text{ dígitos}}$$

donde hay p **dígitos** que se **relacionan** con la **precisión** del número.

Normalización

Decimos que x está **normalizado** si es de la forma:

$$x = \pm \underbrace{d.dd \dots dd}_{p \text{ dígitos}}$$

de modo que haya **un solo dígito distinto de cero** a la **izquierda** del **punto fraccionario**. Para números binarios, si el dígito inicial no es cero, entonces **debe** ser **uno**.

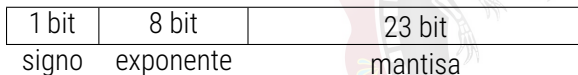
Con el proceso anterior, **no necesitamos almacenar** el **dígito inicial** y, por lo tanto, obtener un poco más de **precisión**; este dígito inicial implícito a menudo se denomina **dígito oculto**.

Formato IEEE-754

Ofrece dos **formatos estándar** que permiten representar números de **punto flotante** como **vectores de bits**. Constan de tres partes: **signo, mantisa y exponente**.

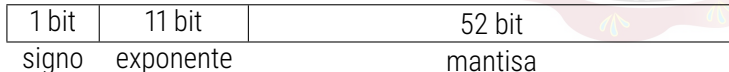
Precisión simple (32 bits)

El **exponente** se almacena en **exceso 127** (se le **suma 127**)



Doble precisión (64 bits)

El **exponente** se almacena en **exceso 1023** (se le **suma 1023**)



Formato IEEE-754

Para representar un **número decimal fraccionario** x al formato **IEEE-754** debemos hacer lo siguiente:

- 1 **Convertimos** x a **binario**.
- 2 **Normalizamos** el resultado dejando un 1 **a la izquierda del punto**.
- 3 **Expresamos** x de la forma $-1^s m \cdot b^e$
- 4 **Calculamos la mantisa** tomando parte fraccionaria de x y rellenando con ceros a la derecha hasta obtener el tamaño deseado.
- 5 **Calculamos el exponente**
- 6 Concatenamos los vectores resultantes y expresamos el resultado en **hexadecimal**.

Formato IEEE-754

Ejemplo

Expresar el 123.3125 en el formato **IEEE-754** de **precisión simple**:

1 **Convertimos** 123.3125 a **binario**:

$$123.3125_{10} = 1111011.0101_2$$

2 **Normalizamos** el número:

$$x = 1. \underbrace{111011}_{6 \text{ posiciones}} 0101_2$$

3 **Expresamos** x de la forma $-1^s m \cdot b^e$ con $s = 0$, $m = 1.1110110101$, $b = 2$ y $e = 6$

$$x = (-1^0) 1.1110110101_2 \cdot 2^6$$

Formato IEEE-754

- 4 **Calculamos la mantisa m** tomando únicamente la **parte fraccionaria** de x y **rellenando con 0's** para obtener un vector de 23 bits:

$$m = \underbrace{11101101010000000000000000000000}_{23 \text{ bits}}$$

- 5 **Calculamos el exponente e** sumándole 127:

$$e = 6 + 127 = 133$$

Lo expresamos en binario:

$$e = 10000101$$

Formato IEEE-754

6 **Concatenamos** los vectores:

0 10000101 111011010100000000000000
 signo exponente mantisa

7 Expresamos el valor en **hexadecimal**:

0 100 0010 1 111 0110 1010 0000 0000 0000
 4 2 F 6 A 0 0 0

8 Finalmente:

$$123.3125 = 0x42F6A000$$

Formato IEEE-754

Ejemplo:

Expresa el -0.5 en formato **IEEE-754** de **precisión simple**:

1 **Convertimos** -0.5 a **binario**:

$$-0.5_{10} = -0.1_2$$

2 **Normalizamos** el número:

$$x = -1. \underbrace{0}_{-1 \text{ posición}}$$

3 **Expresamos** x de la forma $-1^s m \cdot b^e$ con $s = 1$, $m = 1.0$, $b = 2$ y $e = -1$

$$x = (-1^1)1.0_2 \cdot 2^{-1}$$

Formato IEEE-754

- 4 **Calculamos la mantisa m** tomando únicamente la **parte fraccionaria** de x y **rellenando con 0's** para obtener un vector de 23 bits:

$$m = \underbrace{00000000000000000000000}_{23 \text{ bits}}$$

- 5 **Calculamos el exponente e** sumándole 127:

$$e = -1 + 127 = 126$$

Lo expresamos en binario:

$$e = 01111110$$

Formato IEEE-754

6 **Concatenamos** los vectores:

1 01111110 000000000000000000000000
signo exponente mantisa

7 Expresamos el valor en **hexadecimal**:

1 011 1111 0 000 0000 0000 0000 0000 0000 0000
B F 0 0 0 0 0 0 0 0

8 Finalmente:

$$-0.5 = 0xBF000000$$

Formato IEEE-754

Reservamos valores particulares para **representar** $+\infty$, $-\infty$ y **NaN** (not a number). Los valores $+\infty$ y $-\infty$ ocurren cuando un resultado **desborda** los **límites** de lo que se puede **representar**; NaN ocurre, por ejemplo, como resultado de la **división por cero**. En IEEE-754 de **precisión simple**, los valores son:

0	00000000	000000000000000000000000 = $+0$
1	00000000	000000000000000000000000 = -0
0	11111111	000000000000000000000000 = $+\infty$
1	11111111	000000000000000000000000 = $-\infty$
0	11111111	000001000000000000000000 = NaN
1	11111111	00100010001001010101010 = NaN

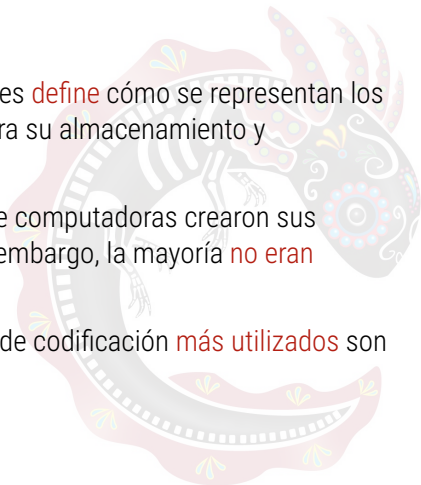
Estándares de codificación

Un **estándar de codificación** de caracteres **define** cómo se representan los **caracteres** en un sistema informático para su almacenamiento y procesamiento.

Durante mucho tiempo los **fabricantes** de computadoras crearon sus **estándares propios** de codificación, sin embargo, la mayoría **no eran compatibles** entre si.

Actualmente, algunos de los estándares de codificación **más utilizados** son los siguientes:

- **ASCII** y sus extensiones
- **UNICODE** (UTF-8, UTF-16 y UTF-32)



ASCII

El código **ASCII (American Standard Code for Information Interchange)** es uno de los **estándares de codificación de caracteres** de los **más antiguos** (creado en 1967) y de los más simples, el cual:

- Se desarrolló originalmente para comunicarse entre los primeros dispositivos de **teleimpresora** y máquinas mecánicas.
- Utiliza una codificación de **7 bits** para representar un juego de **128 caracteres**
- Los primeros **32 caracteres** son de **control**, el resto son **imprimibles**.
- ASCII esta pensado y diseñado para el **idioma inglés**, por lo que no toma en cuenta la **ñ** y las **vocales acentuadas**.

Tabla de caracteres ASCII

H \ L	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	TAB	LF	VT	FF	CR	SO	SI
10	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
20	(SPA)	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
30	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
40	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
50	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
60	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
70	p	q	r	s	t	u	v	w	x	y	z	{		}	~	



Caracteres de control



Caracteres numéricos



Alfabeto latino



Caracteres especiales

Codificación en ASCII

Podemos representar cualquier cadena de caracteres (texto) utilizando el código ASCII, para ello:

- **Separamos** todos los **caracteres** de la cadena.
- Con ayuda de la **tabla ASCII**, **buscamos** el valor en **hexadecimal** de cada carácter y los anotamos debajo de cada carácter.
- Si la cadena contiene **saltos de línea**, estos se representan con los caracteres de control **CR/LF** en **sistemas windows (DOS)** o **LF** en **sistemas UNIX**.
- Una vez codificados la totalidad de los caracteres, **se agrega** al final un **carácter no imprimible** llamado **fin de línea** el cual se representa con el carácter **NUL** o **\0**
- El carácter **fin de línea** que sirve para **indicarle** a la computadora **donde termina** la cadena.

Codificación en ASCII

Ejemplo:

La palabra **Computadora** en ASCII se codifica de la siguiente forma:

C	o	m	p	u	t	a	d	o	r	a	\0
0x43	0x6F	0x6D	0x70	0x75	0x74	0x61	0x64	0x6F	0x72	0x61	0x00

Ejemplo:

Codifique la siguiente cadena de texto en ASCII

Hola!
Saludos.

La cadena codificada en **sistemas windows (DOS)** es:

H	o	l	a	!	CR	LF	S	a	l	u	d	o	s	.	\0
0x48	0x6F	0x6C	0x61	0x21	0x0D	0x0A	0x53	0x61	0x6C	0x75	0x64	0x6F	0x73	0x2E	0x00

Ejercicios

Codifique las siguientes cadenas de caracteres utilizando el formato **ASCII**:

Ejercicio 1

Arquitectura de computadoras

Ejercicio 2

METRO Constitucion de 1917

Ejercicio 3

UACM

MINAS

AV. DEL ARBOL

Ejercicio 4

Juan!

Responde!!

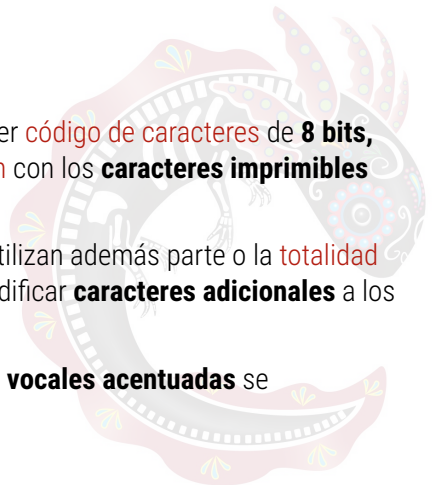


ASCII extendido

Se denomina **ASCII extendido** a cualquier **código de caracteres** de **8 bits**, en el cual los códigos 32 a 126 **coinciden** con los **caracteres imprimibles** de **ASCII**.

Las codificaciones de **ASCII extendido** utilizan además parte o la **totalidad** de los códigos **superiores** a 128 para codificar **caracteres adicionales** a los **caracteres imprimibles** ASCII.

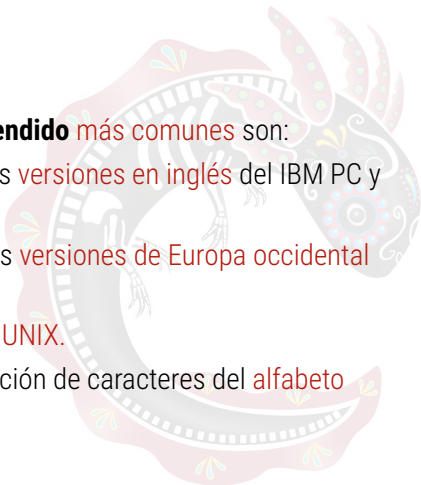
Algunos caracteres como la **letra ñ** y las **vocales acentuadas** se encuentran dentro de estos códigos



Codificaciones ASCII extendido

Algunas de las codificaciones **ASCII extendido** más comunes son:

- **Página de códigos 437**: usual en las versiones en inglés del IBM PC y MS-DOS.
- **Página de códigos 850**: usual en las versiones de Europa occidental del IBM PC y MS-DOS.
- **Latin-1 (ISO/IEC 8859-1)**: típico de UNIX.
- **Windows-1252 (CP-1252)**: codificación de caracteres del alfabeto latino.



Página de códigos 437

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F	
0-		☉	☿	♥	♠	♣	♠	•	▣	○	♠	♂	♀	♪	♪	☀	0-
1-	▶	◀	↕			§	—	↑	↑	↓	→	←	L	↔	▲	▼	1-
2-		!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	2-
3-	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?	3-
4-	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	4-
5-	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_	5-
6-	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	6-
7-	p	q	r	s	t	u	v	w	x	y	z	{		}	~	△	7-
8-	Ç	ü	é	â	ä	à	å	ç	ê	ë	è	ï	î	í	Ä	Å	8-
9-	É	æ	Æ	ô	ö	ò	û	ù	ý	ÿ	Ö	Ü	ø	£	¥	₣	9-
A-	á	í	ó	ú	ñ	Ñ	ª	º	¿	¬	¬	½	¼	¿	«	»	A-
B-	☐	☐	☐														B-
C-	L	⊥	T	⊥	—	+	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	C-
D-	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	⊥	D-
E-	α	β	Γ	π	Σ	σ	μ	τ	Φ	Θ	Ω	δ	∞	φ	ε	∩	E-
F-	≡	±	≥	≤			÷	≈	°	.	.	√	n	²	■		F-
	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F	



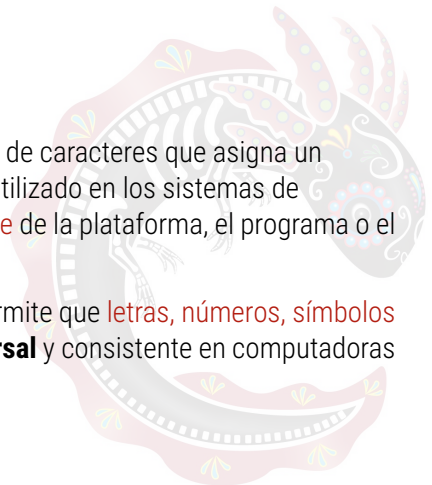
Windows-1252

8-	€ 0080 128*		¡ 201A 130*	¢ 0192 131*	£ 201E 132*	… 2026 133*	† 2020 134*	‡ 2021 135*	ˆ 02C6 136*	‰ 2030 137*	Š 0160 138*	‹ 2039 139*	ƒ 0152 140*		Ž 017D 142*	
9-		€ 145*	¡ 146*	¢ 147*	£ 148*	… 149*	† 150*	‡ 151*	ˆ 02DC 152*	‰ 2122 153*	Š 0161 154*	‹ 203A 155*	ƒ 0153 156*		Ž 017E 158*	Ÿ 0178 159*
A-	nbsp 00A0 160	í 00A1 161	í 00A2 162	£ 00A3 163	¤ 00A4 164	¥ 00A5 165	¦ 00A6 166	§ 00A7 167	¨ 00A8 168	© 00A9 169	ª 00AA 170	« 00AB 171	¬ 00AC 172	SHY 00AD 173	® 00AE 174	¯ 00AF 175
B-	° 00B0 176	± 00B1 177	² 00B2 178	³ 00B3 179	´ 00B4 180	µ 00B5 181	¶ 00B6 182	· 00B7 183	¸ 00B8 184	¹ 00B9 185	º 00BA 186	» 00BB 187	¼ 00BC 188	½ 00BD 189	¾ 00BE 190	¿ 00BF 191
C-	À 00C0 192	Á 00C1 193	Â 00C2 194	Ã 00C3 195	Ä 00C4 196	Å 00C5 197	Æ 00C6 198	Ç 00C7 199	È 00C8 200	É 00C9 201	Ê 00CA 202	Ë 00CB 203	Ì 00CC 204	Í 00CD 205	Î 00CE 206	Ï 00CF 207
D-	Ð 00D0 208	Ñ 00D1 209	Ò 00D2 210	Ó 00D3 211	Ô 00D4 212	Õ 00D5 213	Ö 00D6 214	× 00D7 215	Ø 00D8 216	Ù 00D9 217	Ú 00DA 218	Û 00DB 219	Ü 00DC 220	Ý 00DD 221	Þ 00DE 222	ß 00DF 223
E-	à 00E0 224	á 00E1 225	â 00E2 226	ã 00E3 227	ä 00E4 228	å 00E5 229	æ 00E6 230	ç 00E7 231	è 00E8 232	é 00E9 233	ê 00EA 234	ë 00EB 235	ì 00EC 236	í 00ED 237	î 00EE 238	ï 00EF 239
F-	ø 00F0 240	ñ 00F1 241	ò 00F2 242	ó 00F3 243	ô 00F4 244	õ 00F5 245	ö 00F6 246	÷ 00F7 247	ø 00F8 248	ù 00F9 249	ú 00FA 250	û 00FB 251	ü 00FC 252	ý 00FD 253	þ 00FE 254	ÿ 00FF 255

UNICODE

Unicode es un **estándar de codificación** de caracteres que asigna un número único (código) a cada símbolo utilizado en los sistemas de escritura **del mundo, independientemente** de la plataforma, el programa o el idioma.

En otras palabras, es un sistema que permite que **letras, números, símbolos y emojis** se representen de **forma universal** y consistente en computadoras y dispositivos.



Características de UNICODE

- **Universalidad:** incluye casi todos los **sistemas de escritura**: latín, árabe, chino, griego, cirílico, etc.
- **Compatibilidad con múltiples lenguajes y símbolos:** Soporta **letras**, **acentos**, **ideogramas**, **signos matemáticos**, **símbolos técnicos** y hasta **emojis**.
- **Código único por carácter:** cada carácter tiene un número asignado llamado **punto de código**.

	A	ᄀ	好	不
Punto de código	U+0041	U+05D0	U+597D	U+23B4
UTF-8	41	D7 90	E5 A5 BD	F0 A3 8E B4
UTF-16	00 41	05 D0	59 7D	D8 4C DF B4
UTF-32	00 00 00 41	00 00 05 D0	00 00 59 7D	00 02 33 B4

Formatos de codificación

Los **formatos de codificación** que se pueden usar con UNICODE se denominan **UTF-8, UTF-16 y UTF-32**.

- **UTF-8** utiliza **1 byte** para representar caracteres en el set **ASCII**, **2 bytes** para caracteres en **otros bloques** alfabéticos y **tres bytes** para el **resto** del BMP. Para los **caracteres complementarios** se utilizan **4 bytes**.
- **UTF-16:** utiliza **2 bytes** para **cualquier carácter** en el BMP y **4 bytes** para los **caracteres complementarios**.
- **UTF-32:** emplea **4 bytes** para **todos los caracteres**.

Bibliografía

- 
- [1] M Morris Mano. *Arquitectura de computadoras*. Prentice Hall, 1994.
 - [2] Daniel Page. *A practical introduction to computer architecture*. Springer Science & Business Media, 2009.
 - [3] Andrew S Tanenbaum. *Structured computer organization*. Pearson Education India, 2016.
 - [4] Andrew S Tanenbaum, James R Goodman et al. *Organización de computadoras: un enfoque estructurado*. México: Prentice Hall Hispanoamericana, 1992.